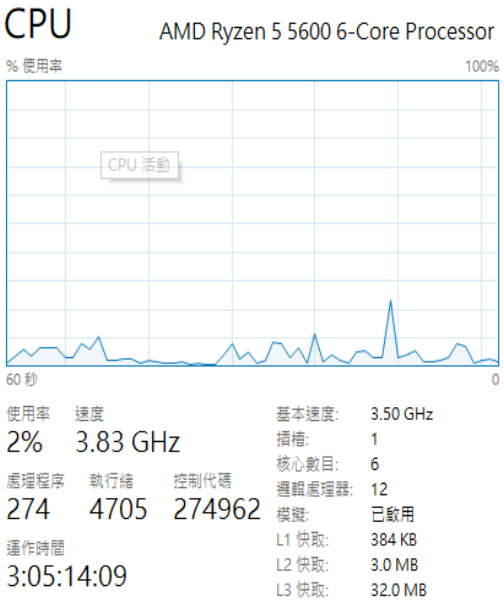
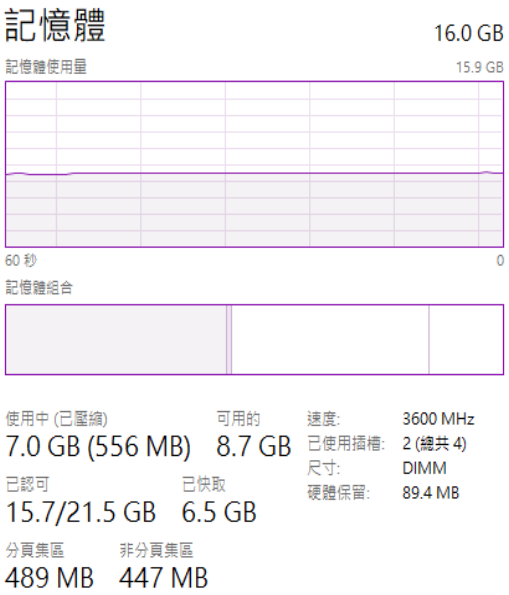
使用編譯器(與其版本)

編譯器	PyCharm Community Edition 2023.1
Python 版本	3.11
作業系統版本:	win 10.0.19045

執行時硬體規格(工作管理員介面):





執行時硬體規格(函數呼叫介面):

透過以下函數的呼叫，來讀取操作環境的訊息

```
import platform
import psutil

print('執行緒:',psutil.cpu_count(),'核')
print('記憶體:',round((psutil.virtual_memory().total/(1024 ** 3))),'GB')

print(platform.platform())
```

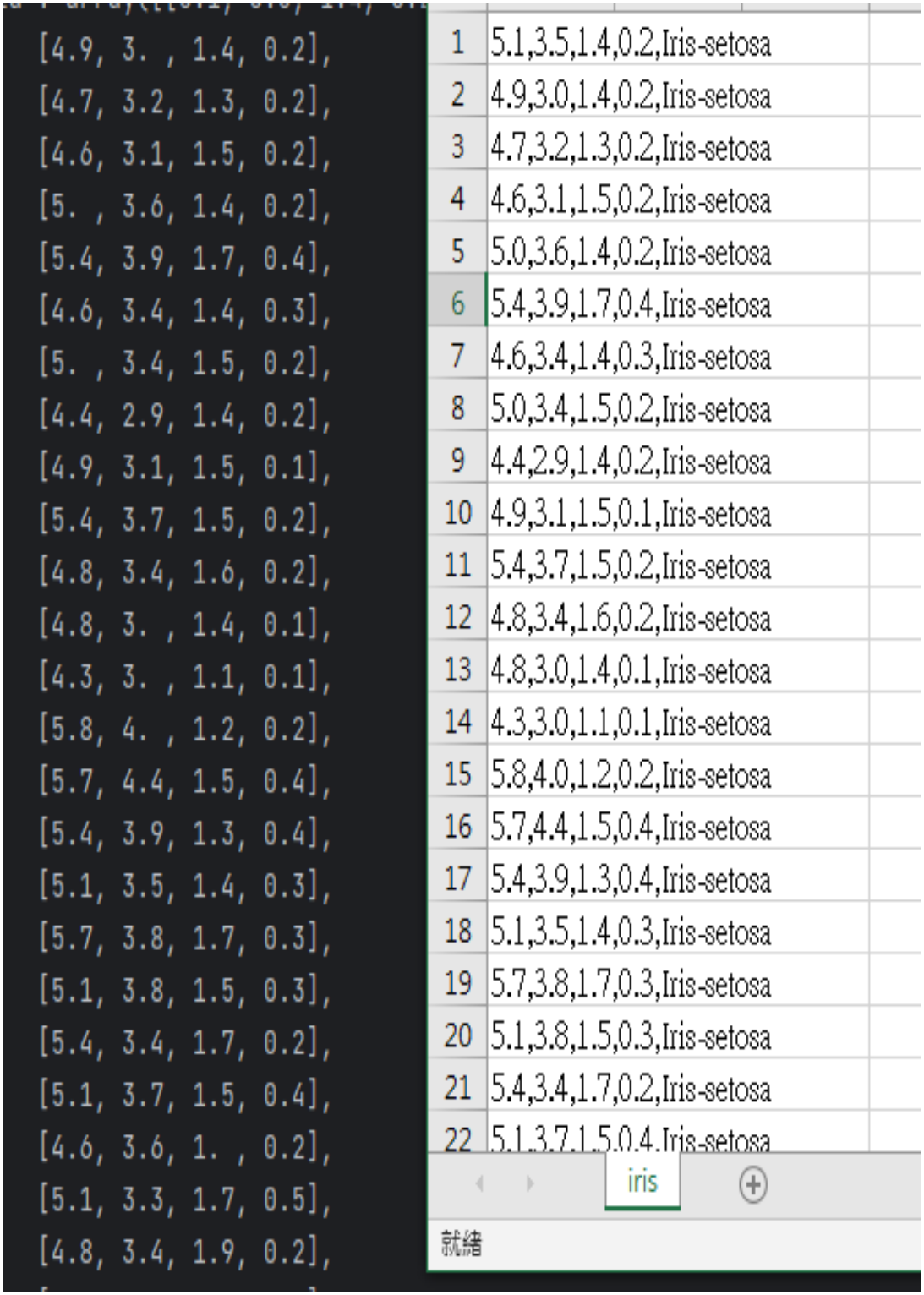
操作環境狀態



資料集:

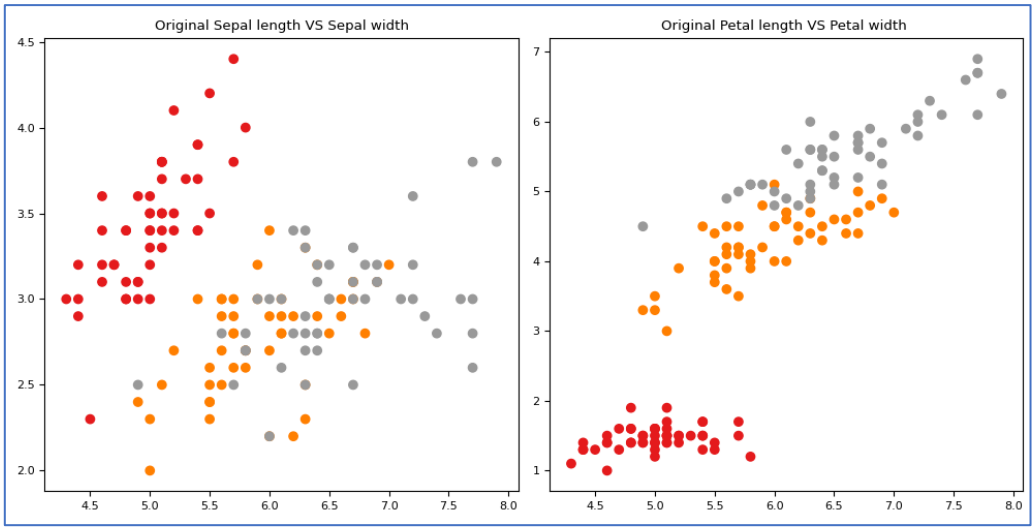
資料集來源(右圖)https://archive.ics.uci.edu/ml/datasets/iris

資料集來源(左圖) Sklearn 模組自帶的 鳶尾花資料集

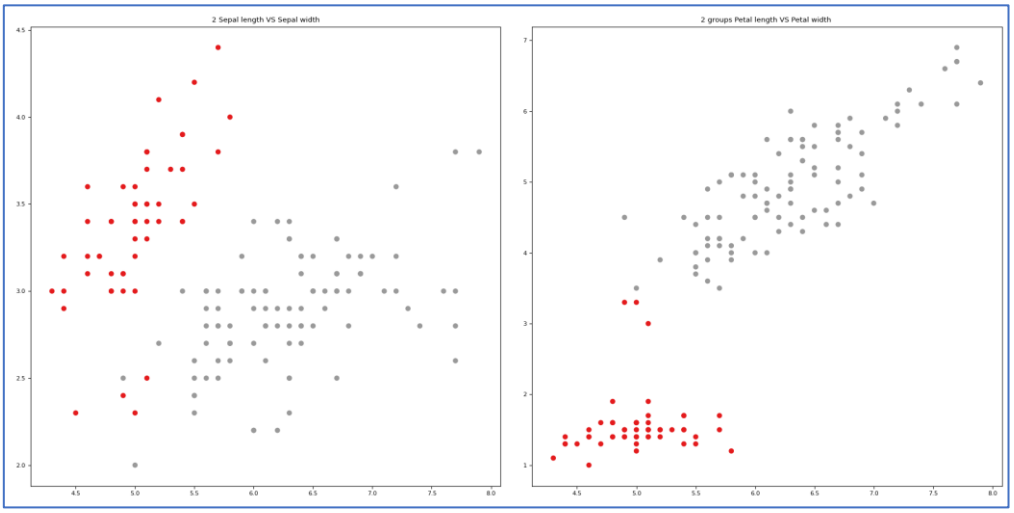


比較不同的分群數對於解讀 Iris Dataset 的影響

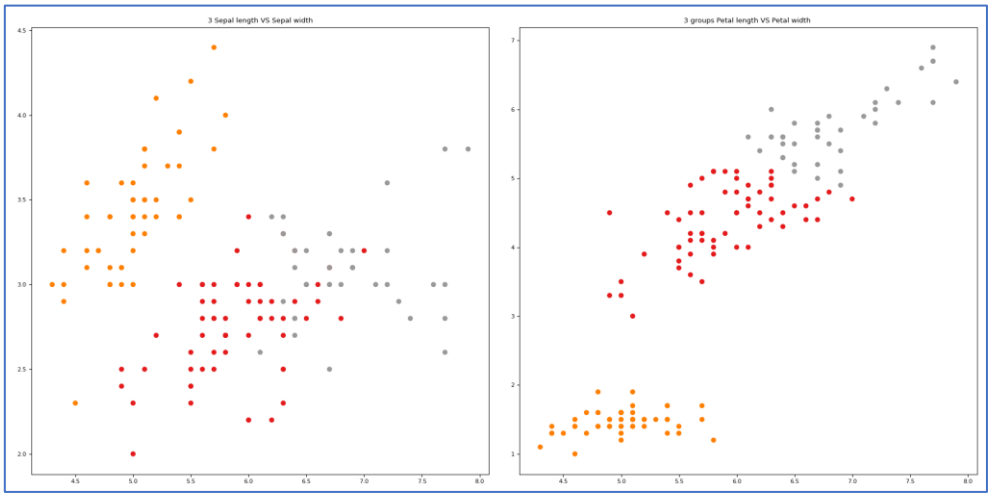
原始----------



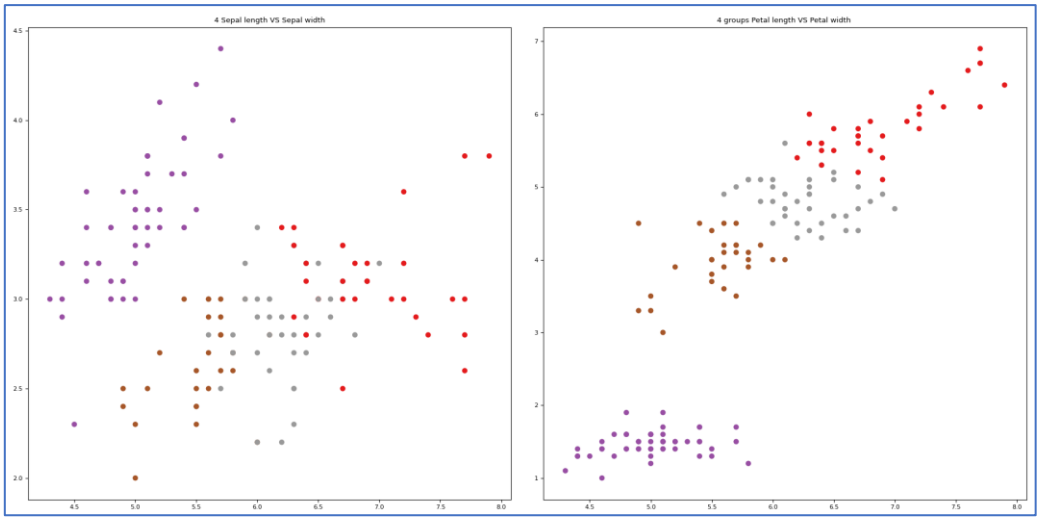
K=2----------



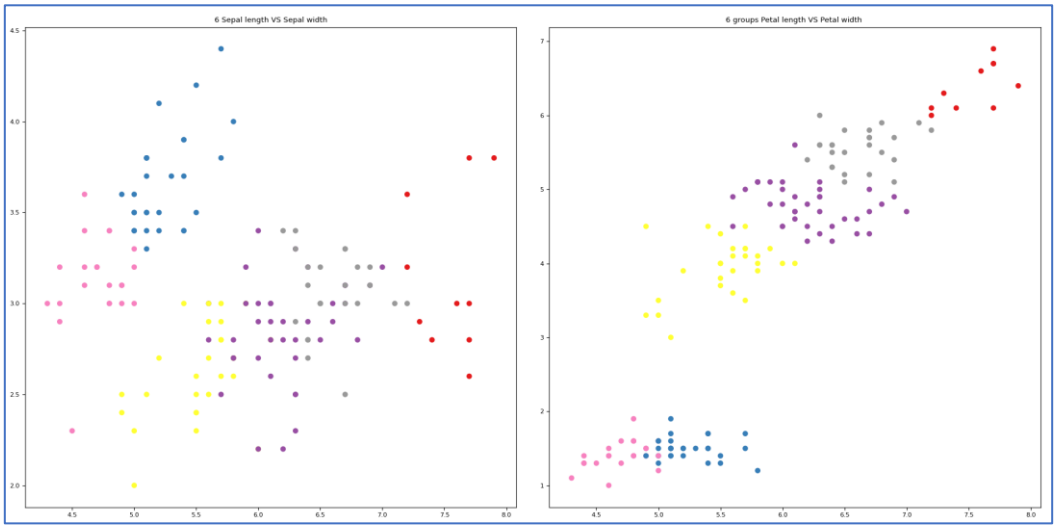
K=3----------



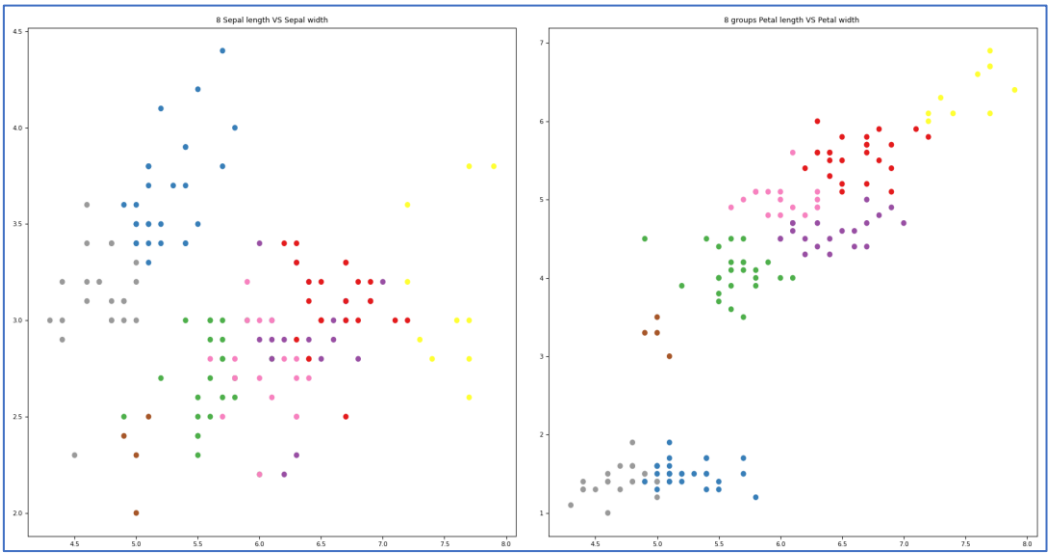
K=4----------


4 Sepal length VS Sepal width
4 groups Petal length VS Petal width

K=6----------


6 Sepal length VS Sepal width
6 groups Petal length VS Petal width

K=8----------


8 Sepal length VS Sepal width
8 groups Petal length VS Petal width

K-means 的運作原理，流程如下:

步驟 1. 首先，我們要先決定想要把樣本分成幾類，比方說 K 類

步驟 2. 隨機給定 K 個中心點（虛擬點，非樣本點）

步驟 3. 計算每個樣本與每個中心點之間的距離，劃分與最近的中心點為一群

步驟 4. 用每個樣本的座標來計算每群樣本的新中心點，並以新中心點取代舊中心點

步驟 5. 重複步驟 3 與 4，直到中心點不再變動為止

從原理可以注意到，每個樣本都會被納入計算，因此最終中心點非常容易受到極值的影響，導致分群的效果可能有落差，不過在此份資料集內，一樣在分三類的情況下去做預測輸出仍有 9 成以上的準確率。

根據上述幾張輸出同型的情況，圖上的極值、群與群邊界上的值非常容易在下一次的分群被劃分成不同群組。由於鳶尾花的資料集數量較少(150 筆)，並且原始的資料依照原始的品種的萼片參數做分群時(原始分布圖.左)本身就有兩個品種有大量的重疊的區域，我認為在兩個品種在這兩個特徵區間的比較下相對模糊，不容易劃出一條明顯線做差別，而後續隨著分群的提高也可以發現這個區域分群的分布情況非常混亂不夠清晰。

相反的，花瓣參數做分群時(原始分布圖.右)，由於本身的特徵足夠明顯，所以分群的狀況對比花萼顯的乾淨俐落，幾乎沒有灰色區間的問題，通常直接看圖都能大概猜出可能會分成哪幾群。並且即便分群數增加，群與群的界線仍相當分明；但是正因為特徵足夠明顯，因此花瓣的資料在做分群時需要留意極值的數量及預處理，否則極值非常容易出現判錯的情況，從原始圖挑幾個極值做觀察，誤判的情況非常明顯。這是由於 K-means 方法的特性，脫離大部隊的極值，幾乎一定會離原始或者新的中心點都太過遙遠。

提高分群數量的時候，分群的現象多半會發生在比較大的群裡面；比較小的群組除了在邊界上會有一點變化之外，大致上不會有變動。實驗上，我是選擇比較:花瓣的長寬、花萼的長寬，兩組做對比，由於有品種的標籤我們可以先從分布圖去評估適不適合分群。從多張不同分群的比較圖當中，可以發現一個現象，特徵明顯的圖(花瓣長寬)在做分群時界線相對明顯，但用與品種數量相同的分類量時，極值誤判的情況相當嚴重；特徵重疊的圖(花萼長寬)在做分群時，界線雖然很模糊，但反而在分類數量相同的情況下，極值誤判的情況要比另一方還要少很多。

從實驗結果探討，K-means 方法應用在資料分群上的效果

該選擇何種分群數量作為最終結果，我認為是很重要的部分，不一定每次做分群的時候都能事先知道有幾個品種做為評估依據，因此要選用何種數量作為依據，我選擇透過 SEE 評估法及輪廓分析法作為最優解的選擇依據。

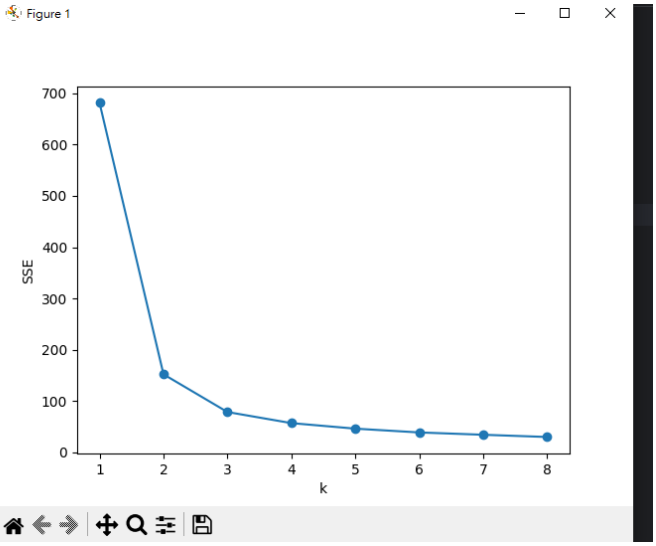
引用自:CSDN SSE

```python
from sklearn import datasets
from sklearn.datasets import load_iris

#鳶尾花資料集
iris_dataset=load_iris()
iris = datasets.load_iris()

dx = iris_dataset['data']
dy = iris_dataset['target']

'''
k-means最优K的选取
'''
'针对对特征Parameter1，利用SSE选择最优k'
SSE = []  # 存放每次结果的误差平方和
for k in range(1, 9):  # K的范围 : 1-9
    estimator = KMeans(n_clusters=k)  # 构造聚类器
    estimator.fit(dx)
    SSE.append(estimator.inertia_)
X = range(1, 9)
plt.xlabel('k')
plt.ylabel('SSE')
plt.plot(X, SSE, 'o-')
plt.show()
```

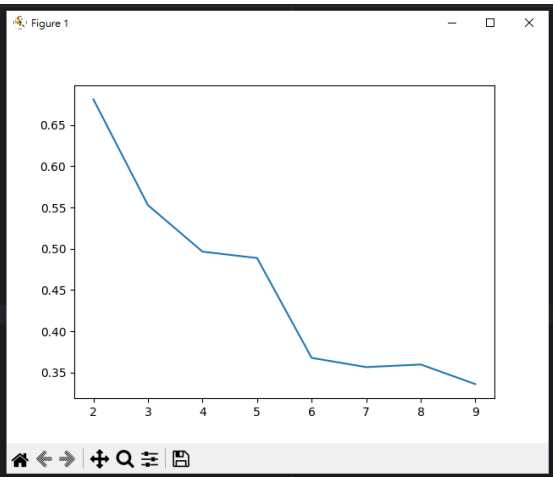


引用自:CSDN 輪廓

```python
from sklearn.metrics import silhouette_score
import pandas as pd
from sklearn import datasets
import matplotlib; matplotlib.use('TkAgg')
from sklearn.datasets import load_iris
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt
import matplotlib; matplotlib.use('TkAgg')
# 定义KMeans，以及K值

iris_dataset=load_iris()
iris = datasets.load_iris()

dx = iris_dataset['data']
dy = iris_dataset['target']
scores=[]
for i in range(2,10):
    kmeans = KMeans(n_clusters=i)
    result_list = kmeans.fit_predict(dx)
    score = silhouette_score(dx, result_list)
    scores.append(score)
plt.plot(range(2,10),scores)
print(pd.DataFrame(scores).idxmax()[0]+2) #轮廓系数越大越好（最大是1）
plt.show()
```



- SSE 的方案會以斜率最大的終點作為選擇依據
- 輪廓分析法則是以最接近 1 的為主

分析結果，理想 K 應等於 2

通過將資料分為 K 個不同的群集，以最小化每個群集內部點之間的歐幾里得距離。儘管 K-means 可以在許多情況下產生良好的結果並且容易實現，但透過原理可以發現，它並不保證在所有情況下都會表現良好。

實驗裡發現幾個可能影響 K-means 方法效果的因素：

初始群集中心的選擇： K-means 是一種基於隨機初始值的算法，不同的初始值可能會導致不同的結果。如果初始值不佳，可能會導致 K-means 無法達到最佳結果。因此有必要在相同的K底下進行多次的產出，以求最佳化的解果。

群集數量的選擇： K-means 需要手動指定群集數量 K，但通常並不知道最佳的 K 值是多少。選擇不當的 K 值可能會導致 K-means 無法達到最佳效果。而我們實際上可以透過鳶尾花的資料集知道他的品種數量，但特別的是在實驗上選擇透過SSE分析及輪廓分析兩種作為依據，皆產生與事實不同的數量，我認為這與極值有關、並且在某些參數下也有很多的灰色地帶，所以導致評估的結果與實際上的品種數量有異，或者品種三本身就不是最佳解。